第4章

体系结构风格

ARCHITECTURAL STYLES



陈龙彪
longbiaochen@xmu.edu.cn

厦门大学信息学院
2019年3月

教学内容

- 定义
- 主程序-子程序风格
- 面向对象风格
- 层次结构风格
- 仓库风格
- 黑板风格
- 隐式调用风格
- 管道-过滤器风格
- C/S和B/S风格







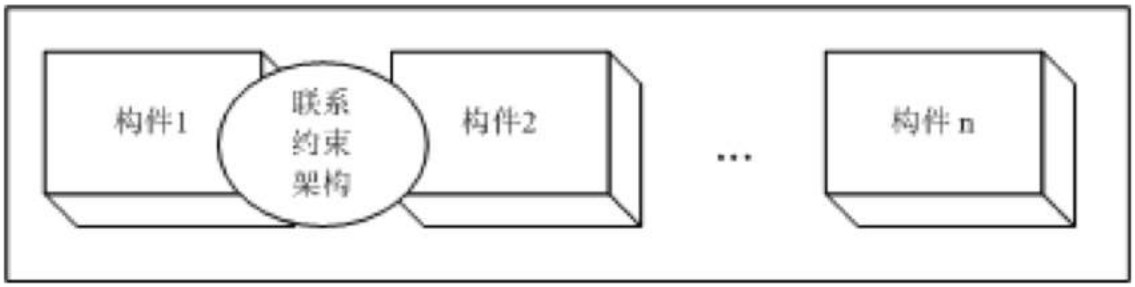


定义

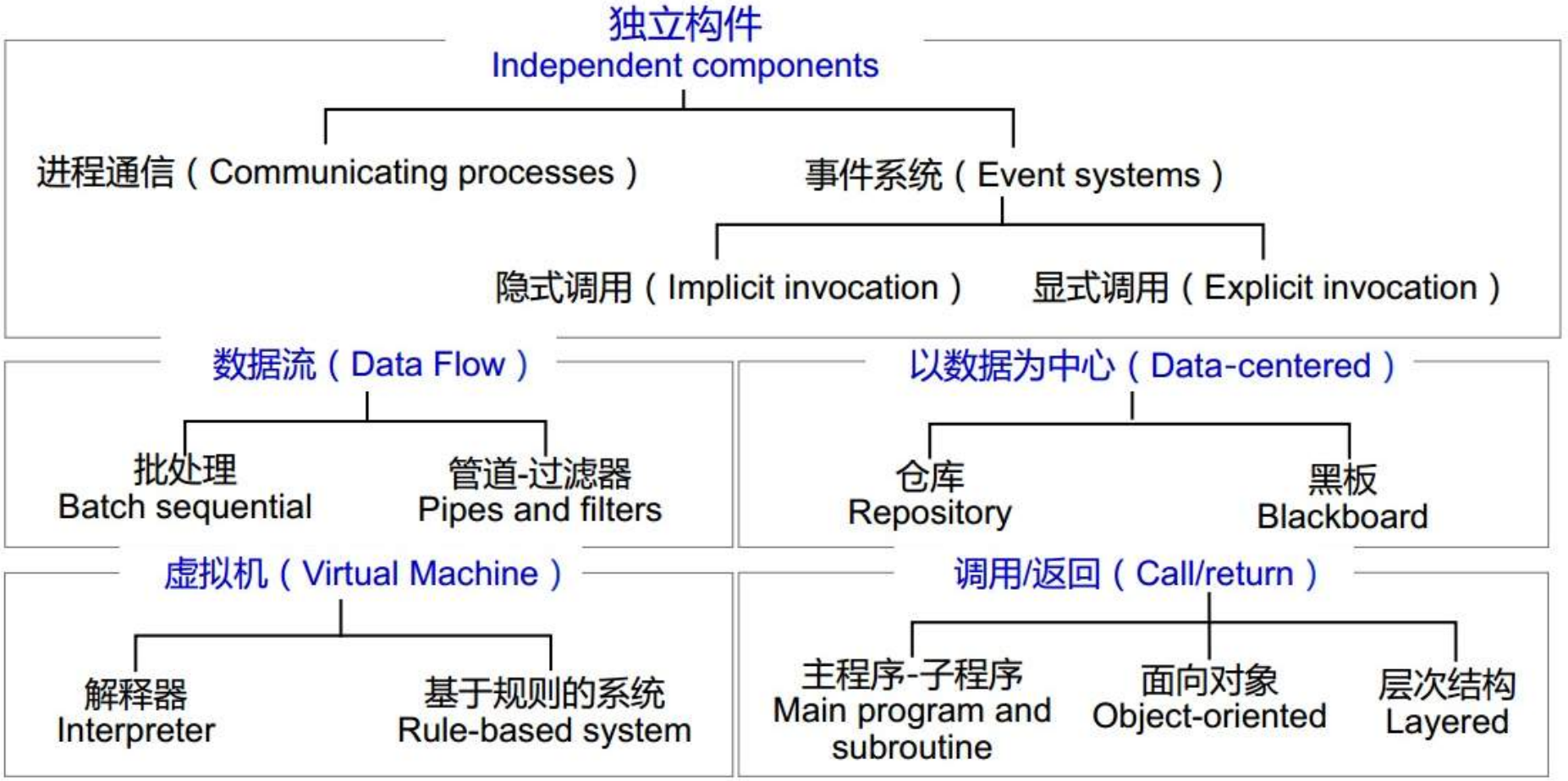
- 软件体系结构由构件和构件之间的联系组成
- 软件体系结构风格是描述某一特定应用领域中系统组织方式的惯用模式
- 体系结构风格定义了一个词汇表和一组约束
  - 词汇表中包含一些构件和连接件类型
  - 约束指出系统是如何将这些构件和连接件组合起来的
- 体系结构风格反映了领域中众多系统所共有的结构和语义特性，并指导如何将各个模块和子系统有效地组织成一个完整的系统

定义

- 软件体系结构主要涉及三方面内容
  - 构件
  - 构件之间的联系与约束
  - 由构件通过相互交互形成的系统架构

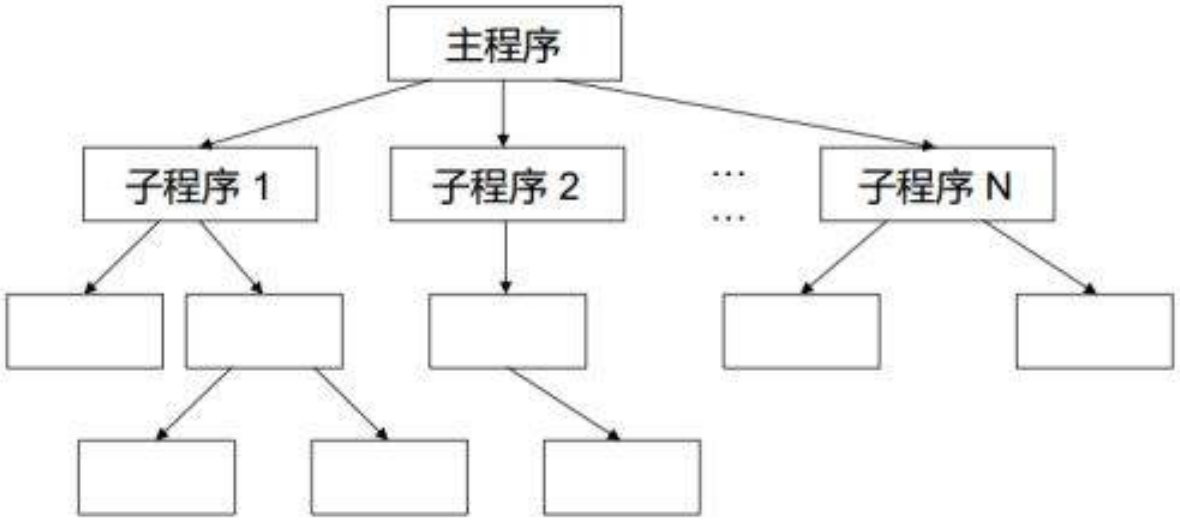


经典的体系结构风格


独立构件
Independent components
进程通信（Communicating processes）
事件系统（Event systems）
隐式调用（Implicit invocation）
显式调用（Explicit invocation）
数据流（Data Flow）
批处理
Batch sequential
管道-过滤器
Pipes and filters
以数据为中心（Data-centered）
仓库
Repository
黑板
Blackboard
虚拟机（Virtual Machine）
解释器
Interpreter
基于规则的系统
Rule-based system
调用/返回（Call/return）
主程序-子程序
Main program and subroutine
面向对象
Object-oriented
层次结构
Layered

主程序-子程序风格

- 定义：从功能的观点设计系统，通过逐步分解和细化，形成整个系统的体系结构



构件：主程序、子程序
连接器：调用-返回机制
拓扑结构：层次化结构

主程序-子程序风格

- 优点:
  - 有效地将一个较复杂的程序系统设计任务分解成许多易于控制和处理的子任务
- 缺点:
  - 规模：程序超过10万行，表现不好；程序太大，开发太慢，可重用性差，测试越来越困难
  - 可扩展性差：当数据结构改变时，所有相关处理过程都要进行相应的修改
  - 图形用户界面的应用程序，很难用过程来描述和实现，开发和维护也都很困难

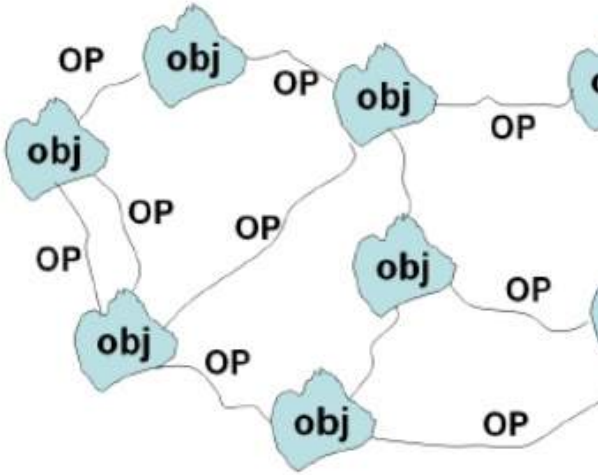
https://blog.csdn.net/linhaiyun_ytdx/article/details/61923670

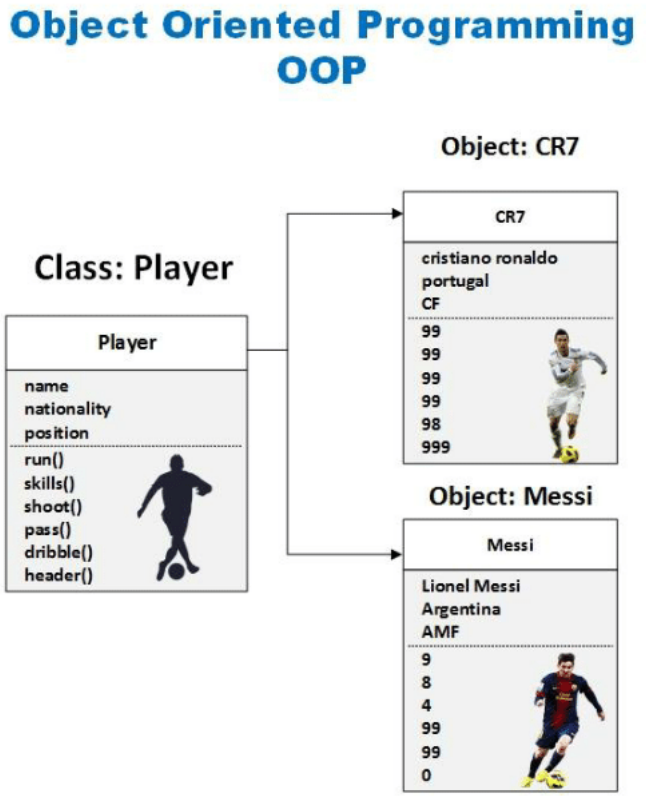
面向对象风格

- 定义：数据的表示方法和它们的相应操作被封装在一个抽象数据类型或对象中
  - 这种风格的构件是对象，或抽象数据类型的实例
  - 对象通过函数和过程的调用来进行交互
  - 系统被看作是对象的集合，每个对象都有一个它自己的功能集合
  - 数据及作用在数据上的操作被封装成抽象数据类型
  - 只通过接口与外界交互，内部的设计决策则被封装起来

面向对象风格

- 结构





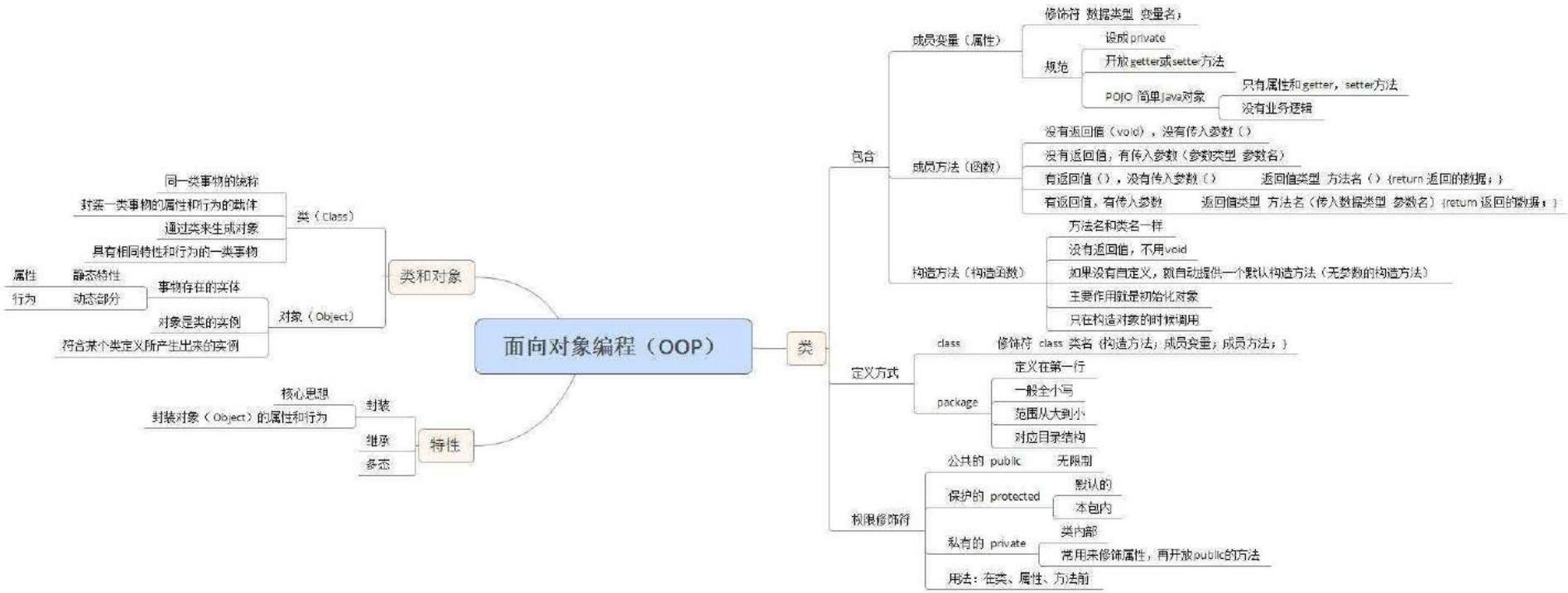


函数调用和消息传递实现交互

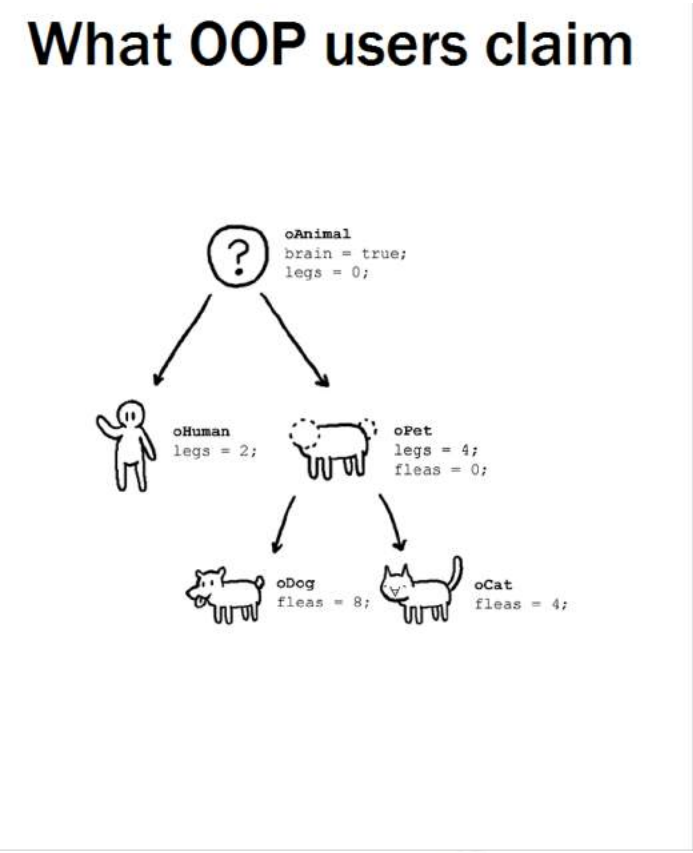
How to learn OOP using football

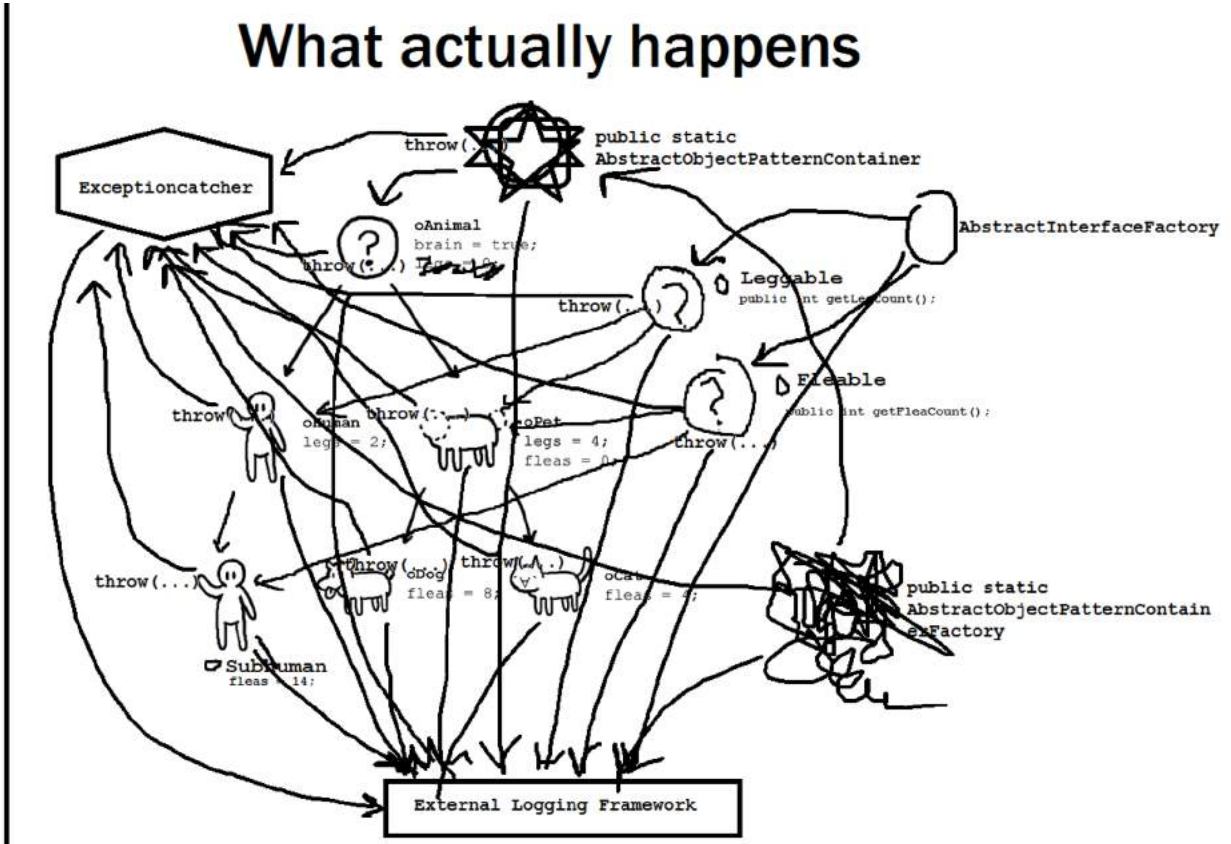
面向对象风格

- 结构


面向对象编程（OOP）
类和对象
类（Class）
同一类事物的统称
封装一类事物的属性和行为的载体
通过类来生成对象
具有相同特性和行为的一类事物
对象（Object）
事物存在的实体
属性
静态特性
行为
动态部分
对象是类的实例
符合某个类定义所产生出来的实例
特性
封装
核心思想
封装对象（Object）的属性和行为
继承
多态
类
包含
成员变量（属性）
修饰符 数据类型 变量名；
规范
设成private
开放getter或setter方法
POJO 简单Java对象
只有属性和getter，setter方法
没有业务逻辑
成员方法（函数）
没有返回值（void），没有传入参数（）
没有返回值，有传入参数（参数类型 参数名）
有返回值（），没有传入参数（） 返回值类型 方法名（）{return 返回的数据；}
有返回值，有传入参数 返回值类型 方法名（传入数据类型 参数名）{return 返回的数据；}
构造方法（构造函数）
方法名和类名一样
没有返回值，不用void
如果没有自定义，则自动提供一个默认构造方法（无参数的构造方法）
主要作用就是初始化对象
只在构造对象的时候调用
定义方式
class 修饰符 class 类名 {构造方法；成员变量；成员方法；}
package
定义在第一行
一般全小写
范围从大到小
对应目录结构
权限修饰符
公共的 public 无限制
保护的 protected 默认的
本包内
私有的 private 类内部
常用来修饰属性，再开放public的方法
用法：在类、属性、方法前

面向对象风格







```
93                    )
94                ),
95                "text/plain"
96            )
97        )
98    )
99),
100   new FkFixed(
```



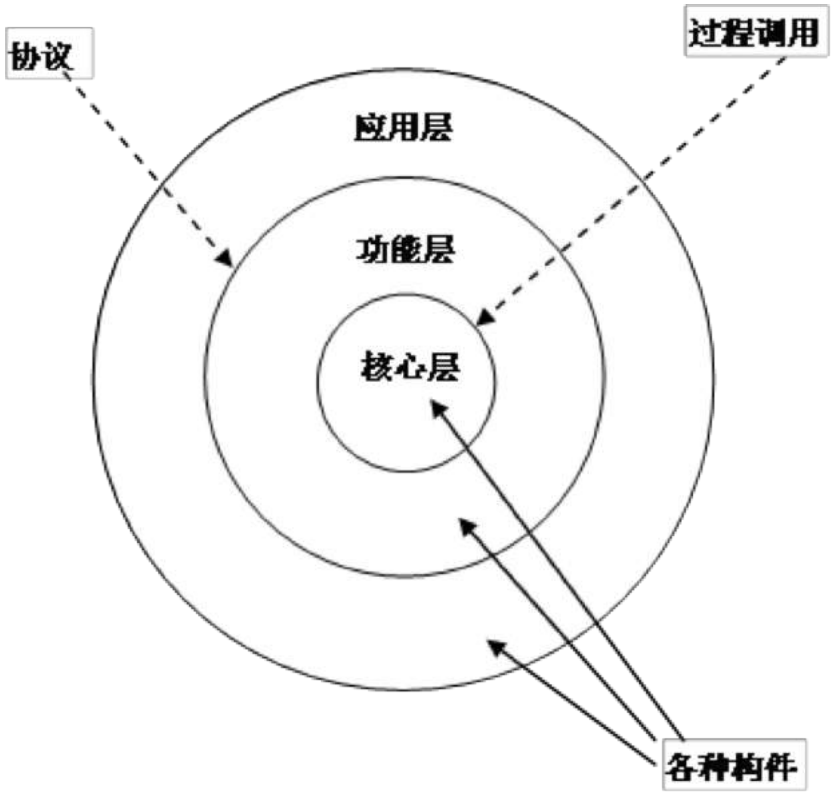
层次结构风格

- 定义：系统组织成层次结构，每一层都为上一层提供服务，并且接受下一层提供的服务
  - 将复杂系统划分为多个独立的层次，每一层都具有高度的内聚性
  - 较高层面向特定应用问题，较低层则更具有一般性
  - 层间连接器通过层间交互的协议来定义，且上下层之间是单向调用关系，即上层通过下层提供的接口来使用下层的功能，而下层却不能使用上层的功能
  - 一些层次中构件实现了虚拟机的功能

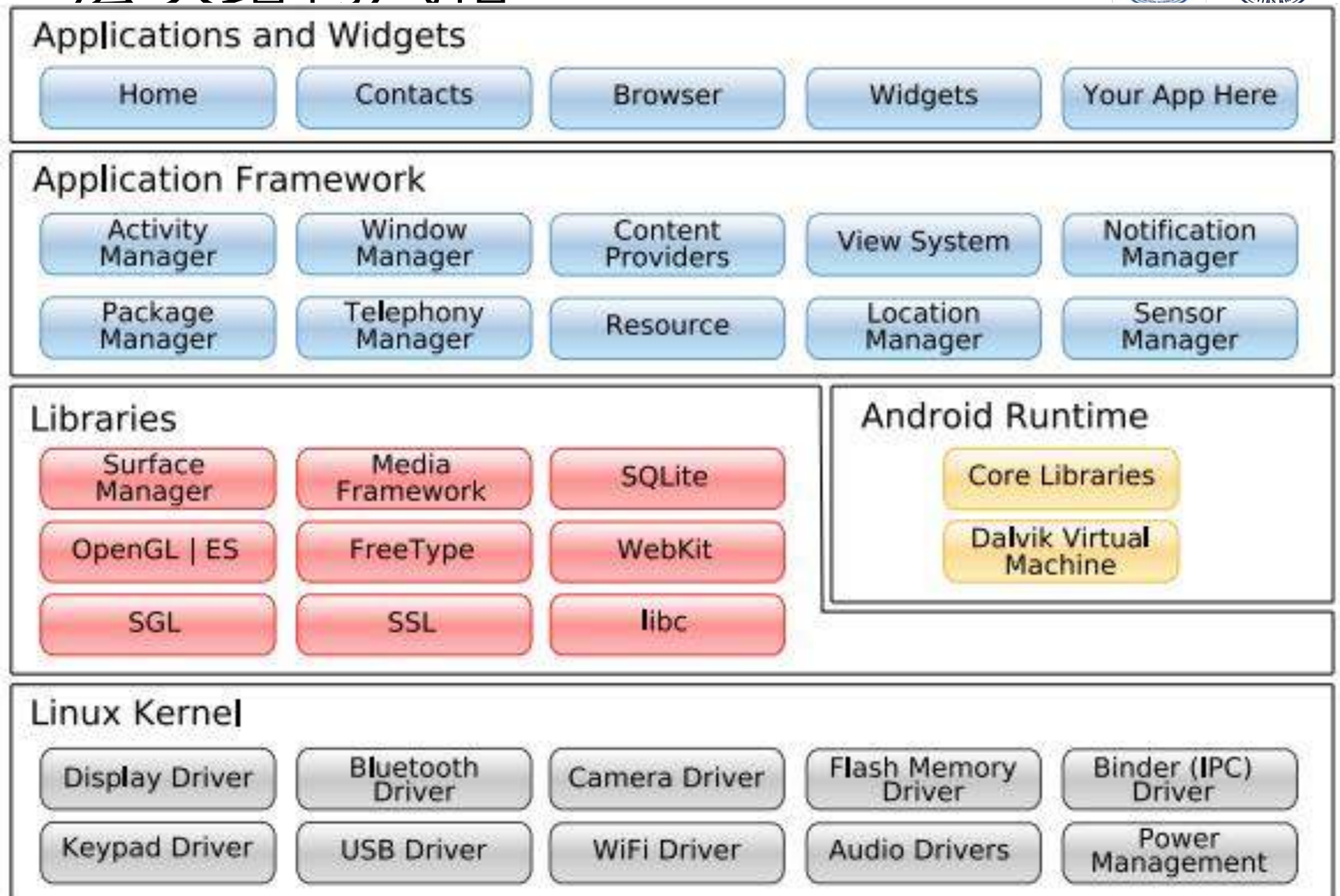
https://blog.csdn.net/linhaiyun_ytdx/article/details/71512181

层次结构风格

- 结构
  - 基本组件：各层次内部包含的组件
  - 连接件：层间的交互协议
  - 拓扑结构：分层



层次结构风格


Applications and Widgets
Home
Contacts
Browser
Widgets
Your App Here
Application Framework
Activity Manager
Window Manager
Content Providers
View System
Notification Manager
Package Manager
Telephony Manager
Resource
Location Manager
Sensor Manager
Libraries
Surface Manager
Media Framework
SQLite
OpenGL | ES
FreeType
WebKit
SGL
SSL
libc
Android Runtime
Core Libraries
Dalvik Virtual Machine
Linux Kernel
Display Driver
Bluetooth Driver
Camera Driver
Flash Memory Driver
Binder (IPC) Driver
Keypad Driver
USB Driver
WiFi Driver
Audio Drivers
Power Management

层次结构风格

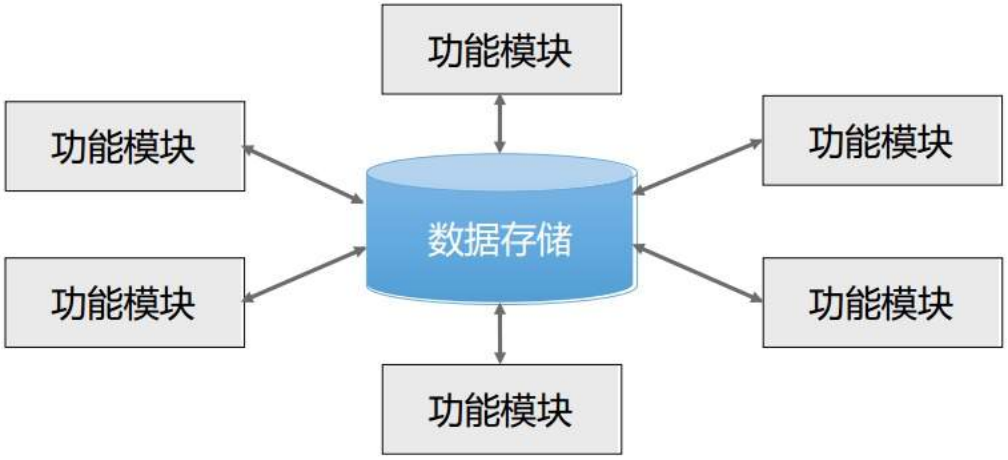
- 优点
  - 支持基于可增加抽象层的设计，允许将一个复杂问题分解成一个增量步骤序列的实现。
  - 可复用性：每一层最多只影响两层，同时只要给相邻层提供相同的接口，允许每层用不同的方法实现
  - 局部依赖性：因此功能的改变通常只影响相邻的上下层
  - 可替换性：可以定义一组标准接口，而允许各层采用各种不同的实现方法
  - 可测试性：具有定义明确的层接口以及交换层接口的各个实现的能力提高了可测试性

层次结构风格

- 缺点
  - 并不是每个系统都可以很容易地划分为分层的模式
  - 效率降低：
    - 在上层中的服务如果有很多依赖于最底层，则相关数据必须通过中间层的若干次转化才能传递到
  - 很难找到一个合适的、正确的层次抽象方法
    - 层数太少：不能完全发挥可复用性、可更改性和可移植性潜力
    - 层数过多：不必要的复杂性、层间隔离冗余、层间传输开销

仓库风格

- 定义：中央数据结构（仓库）和独立构件的集合
  - 中央数据结构说明当前状态
  - 独立构件在中央数据上执行
  - 仓库和外部构件之间产生相互作用



仓库风格

```
UIPasteboard *pasteboard = [UIPasteboard generalPasteboard];
[pasteboard setString:@"往剪贴板写信息"];//设置剪贴板信息
NSLog(@"剪贴板信息 = %@",pasteboard.string);//获取剪贴板信息
```

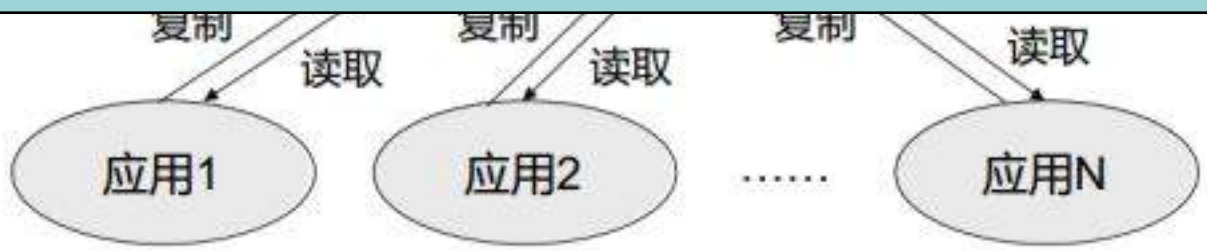
举例：

```
navigator.clipboard.readText()
  .then(text => {
    console.log('Pasted content: ', text);
  })
  .catch(err => {
    console.error('Failed to read clipboard contents:
', err);
  });
```



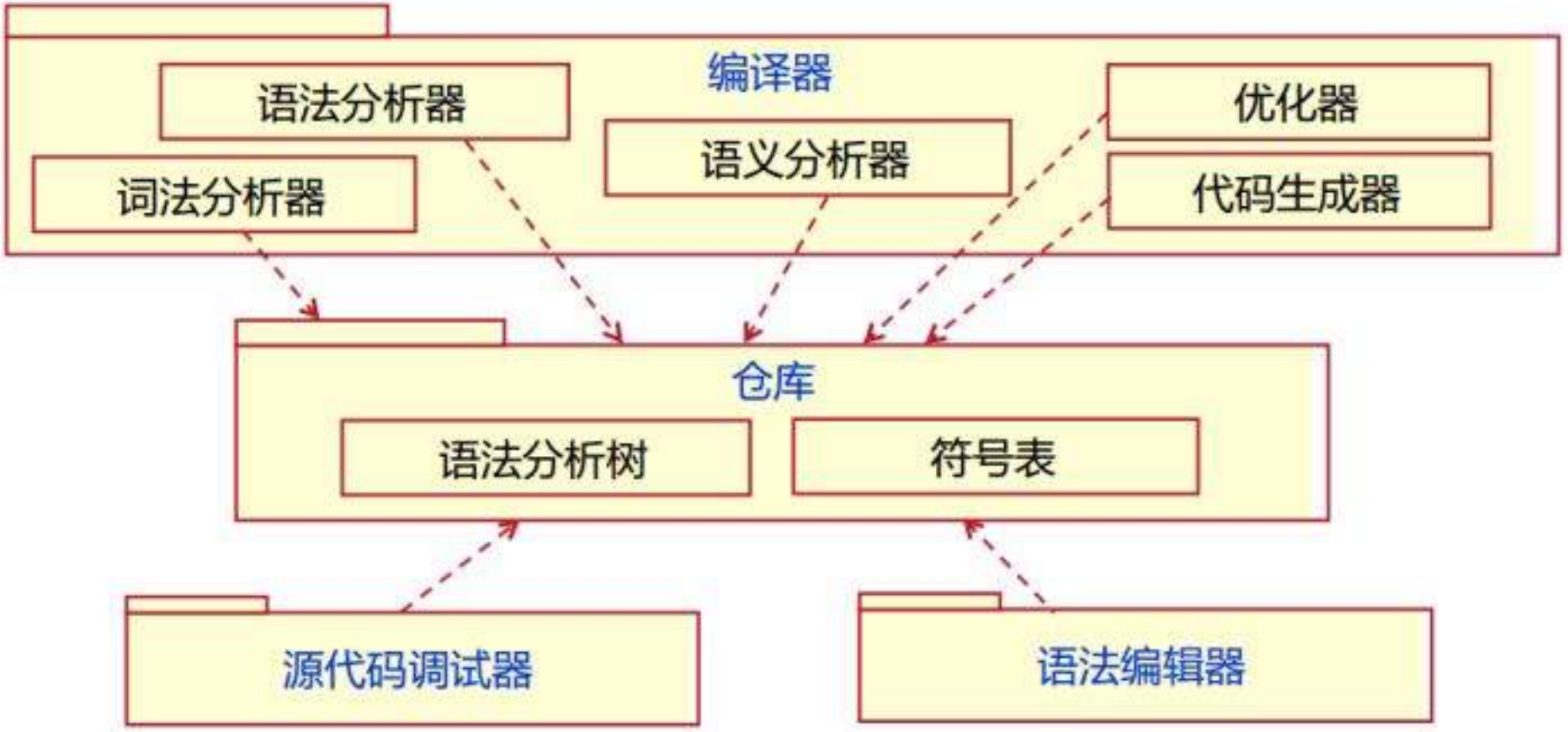
进行

数据传



仓库风格

- 实例2
  - 集成开发环境（IDE）

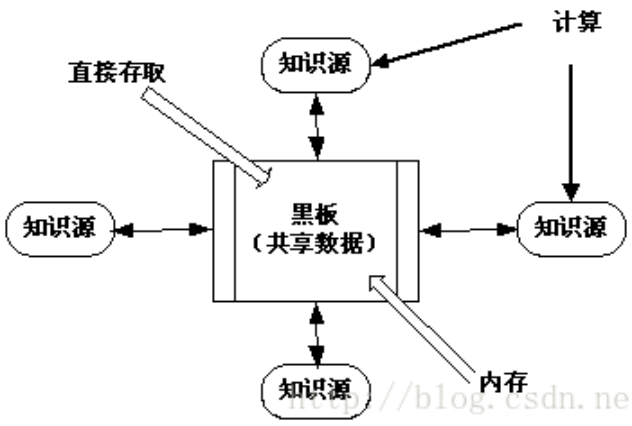


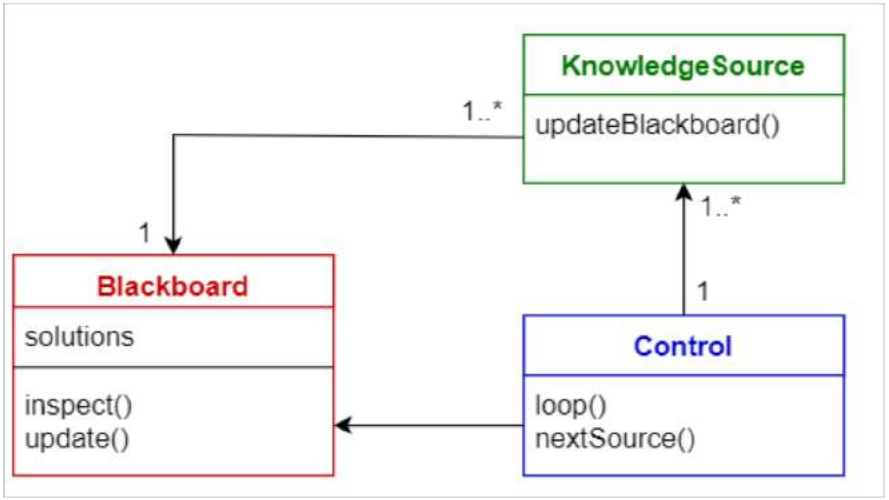
黑板风格

- 引入
  - “黑板”（Blackboard）在人工智能领域已经是一个很古老的东西了，它基于一种很直观的概念，就是一群人为了解决一个问题，在黑板前聚集，每个人都可以发表自己的意见，然后在黑板上写下自己的看法，当然你也可以基于别人记录在黑板上的看法，来发表和更新自己的看法，在这样不断的意见交换，看法更新的过程中，越来越趋向于对于问题的最终解答。一开始的黑板系统就是这样一个由多个子系统来共同协作的人工智能解决方案。

黑板风格

- 定义
  - 知识源：包含独立的、与应用程序相关的知识。知识源之间不直接进行通讯，它们之间的交互通过黑板来完成
  - 黑板数据：按照与应用程序相关的层次来组织的解决问题的数据，知识源通过不断地改变黑板数据来解决问题
  - 控制。控制完全由黑板的状态驱动，黑板状态的改变决定使用的特定知识

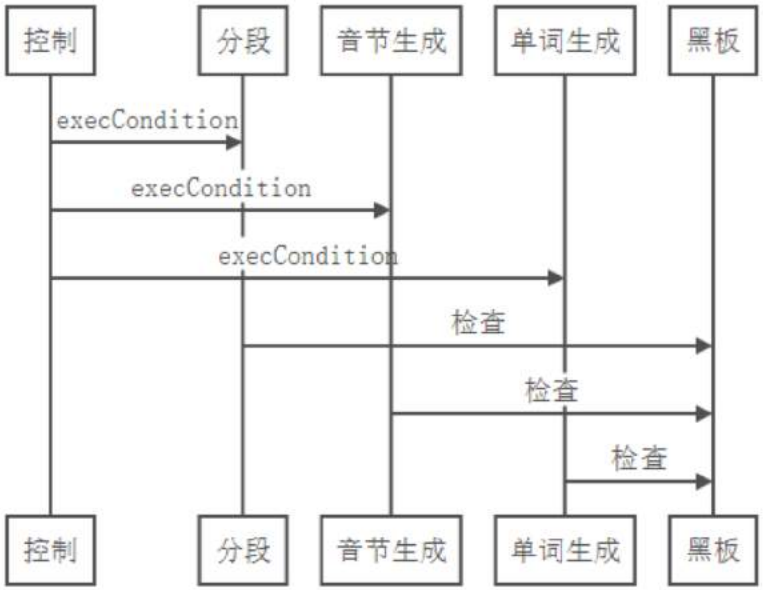




http://blog.csdn.net/

黑板风格

- 实例：黑板系统的典型应用是信号处理领域，如网络信息检索、语音和模式识别、智能控制领域等
  - 利用数据库
  - 利用发布—订阅模式



黑板风格

- 优点
  - 用于
  - 不
  - 不能

黑板是一个共享数据的结构，它对于多个系统间通信是很有帮助的，从程序设计的角度上来说，它提供一种数据传递的方式，有助于系统的封装和解耦合。对于各个子系统而言，只需要把自己的运算的结果数据记录在黑板上，至于这个数据谁会去用，并不需要关心，反过来也是一样，对于自己的运算时需要用到的数据，可以从

既是黄金屋，又是垃圾堆

黑板也有些不足的地方，比如RWD（读写删）操作相对随意，特别是WD操作，容易造成数据被破坏，或者产生子系统间的竞争，比如，系统A和系统B都会去修改data1，那到底以谁的值为准呢？还有一个情况就是产生非法数据，一般认为，只要在黑板上的数据，就是合法的数据，在读取的时候，不需要判断它是否合法，但如果一个子系统没有很好的维护它自己产生的数据（比如，该删除的时候没删除，或者赋值错误），那别人读取该数据的系统时候，就会产生错误的运算结果。


EVE
LEGO
ITALY
UNITED KINGDOM
NIN
MUSE
R/TAGPRO
R/GREENLATTICE
PIRATES
STRAYA
INDIA
R/MMA
R/SCOTLAND
ROCKET LEAGUE
BTS
Gogh.bmp
Start
r/placestart
R.I.P. Tom Searle
r/RBTV

黑板风格

- 例子

2017年的愚人节，欧美最大的社区之一Reddit在/r/place/版块做了一个活动：他们在这个版块的顶部设置了一个1000x1000像素的巨大空白画布，并交由每一名社区成员任意涂抹：每个人可以选择在一个像素点上涂抹一个任意的颜色，但是要等一段时间才可以涂第二次，**“单独的你可以创造一些东西，联合起来你可以创造更多的东西”**。

参与的规则非常简单：所有用户都可以参与；参与者必须遵守Reddit的规定，且不得发布隐私信息，除此之外一切不限。对于游戏玩家而言，这个板块就像一个巨大的沙盒游戏一样，你可以随意发挥你的想象力来创造——你只会受到一个明显的游戏规则制约：**每5／10／20分钟才可以涂一次**。





https://www.zhihu.com/question/57966233

黑板风格

- 例子

Twitch Plays Pokémon（缩写TPP）为一个在Twitch上的直播频道、直播视讯，其直播主、发起人称之为一个“社会学实验”，其内容为一群网友在实况频道的聊天室输入游戏指令来控制直播中的游戏，其游戏主要为任天堂的《精灵宝可梦》游戏系列。

此实验被定位为社会学实验，其系统架构是一位匿名的澳洲程式设计师所设计，包含用Python写成的IRC机器人，以及Game Boy模拟器：Visual Boy Advance。IRC机器人会捕捉使用者在聊天室中输入的字句，并找出特定的字，像是“up”、“down”、“start”等，并将其转为游戏指令传入模拟器中来控制游戏；并额外使用一个用JavaScript写成的网路应用程式来显示其传入了哪些指令。



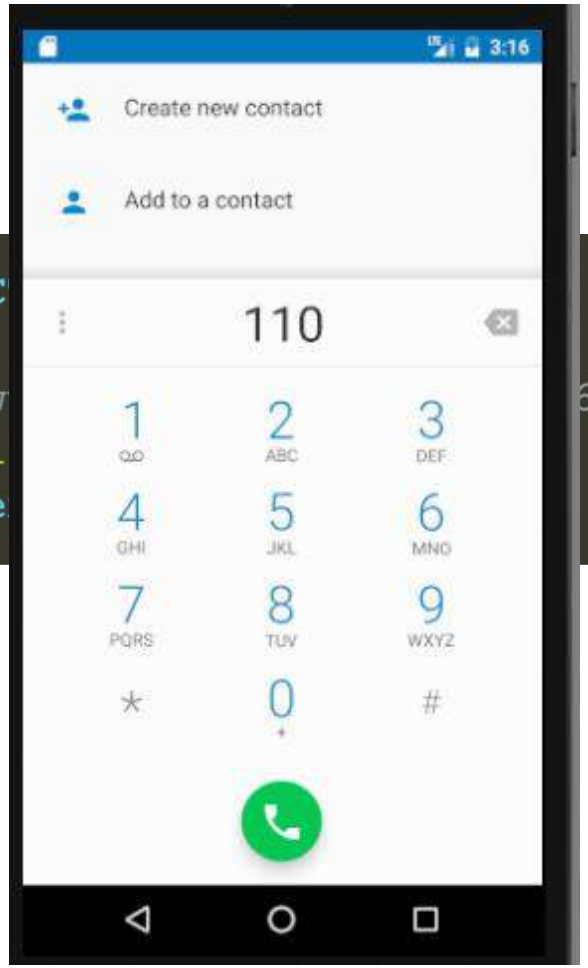
https://zh.wikipedia.org/wiki/Twitch_Plays_Pokémon

隐式调用风格

- 定义：构件不直接调用一个过程，而是触发或广播一个或多个事件
  - 系统中的其他构件中的过程在一个或多个事件中注册，当一个事件被触发，系统自动调用在这个事件中注册的所有过程
  - 这种风格的构件是一个模块，这些模块可以是一些过程也可以是一些事件的集合
  - 事件的触发者并不知道哪些构件会被这些事件影响

隐式调用风格

- 实例
  - 

  - 


```
1.  In                                  ntent.AC
2.
    in                                  ttp://w                    667"));
3.                                      Data(Uri
4.                                      ity(inte
```

https://blog.csdn.net/linhaiyun_ytdx/article/details/66975538

隐式调用风格

- 优点
  - 异步执行：不必同步等待执行结果
  - 并发处理：同时响应将提高系统性能
  - 服务灵活：一个调用可以由多个被调用者响应提供服务
  - 软件重用：当需要将一个构件加入现存系统中时，只需将它注册到系统的事件中
  - 系统改进：当用一个构件代替另一个构件时，不会影响到其它构件的接口

隐式调用风格

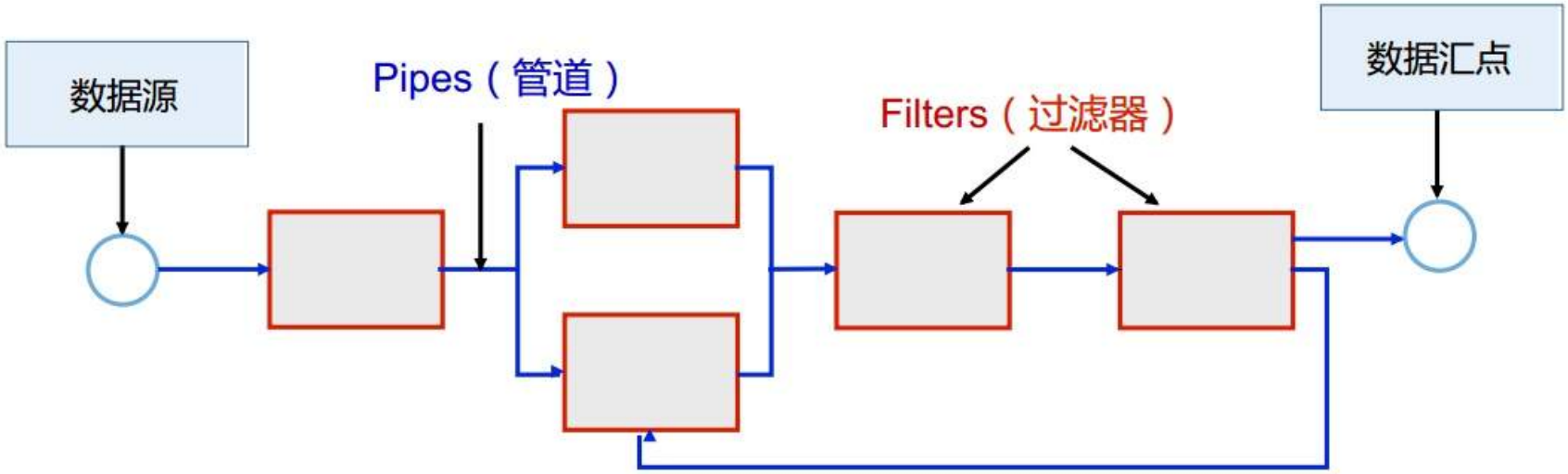
- 缺点
  - 系统控制问题：一个构件触发一个事件时，不能确定其它构件是否会响应它。而且即使它知道事件注册了哪些构件的构成，它也不能保证这些过程被 调用的顺序
  - 数据交换问题：有时数据可被一个事件传递，但另一些情况下，基于事件的系统必须依靠一个共享的仓库进行交互。在这些情况下，全局性能和资源管理便成了问题
  - 推理逻辑问题：既然过程的语义必须依赖于被触发事件的上下文约束，关于正确性的推理存在问题

管道-过滤器风格

- 定义：每个构件都有一组输入和输出，构件读输入的数据流，经过内部处理，然后产生输出数据流
  - 过滤器风格的连接件就象是数据流传输的管道，将一个过滤器的输出传到另一个过滤器的输入。
  - 过滤器虽然可以增量式地处理数据，但是它们是独立的
  - 管道和过滤器的正确输出不依赖其顺序

管道-过滤器风格

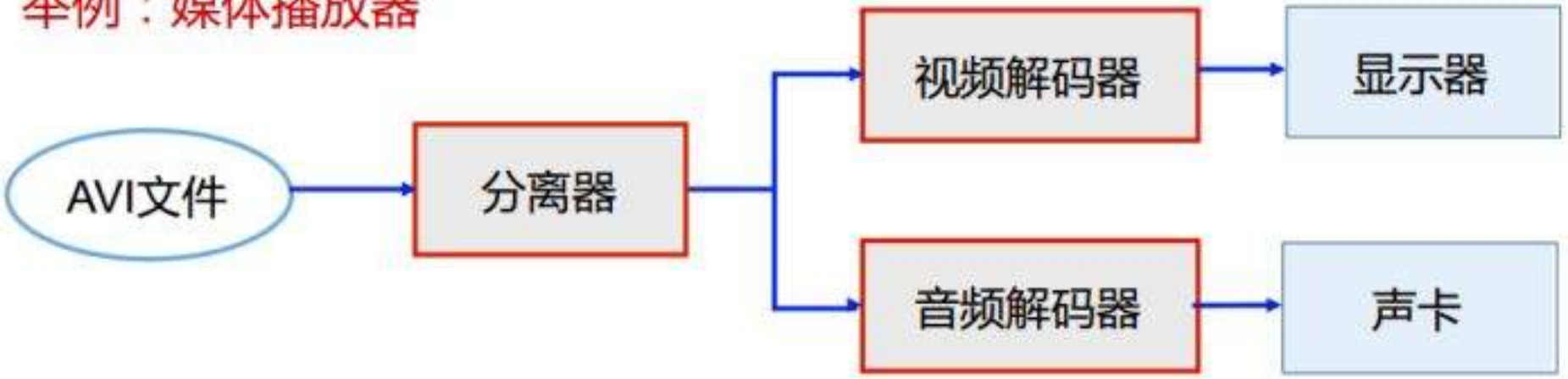
- 结构

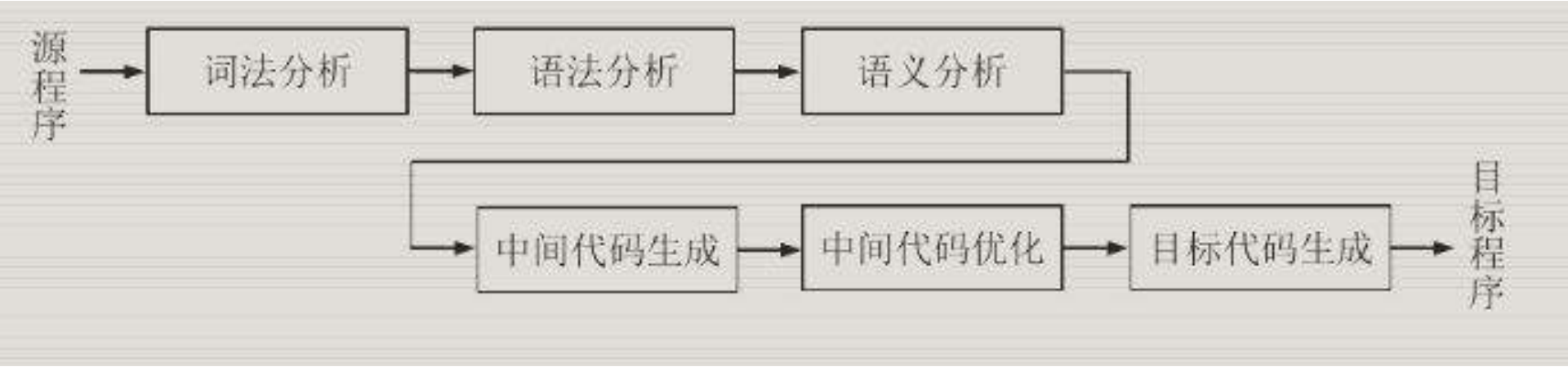


https://blog.csdn.net/linhaiyun_ytdx/article/details/66975538

管道-过滤器风格

举例：媒体播放器





管道-过滤器风格

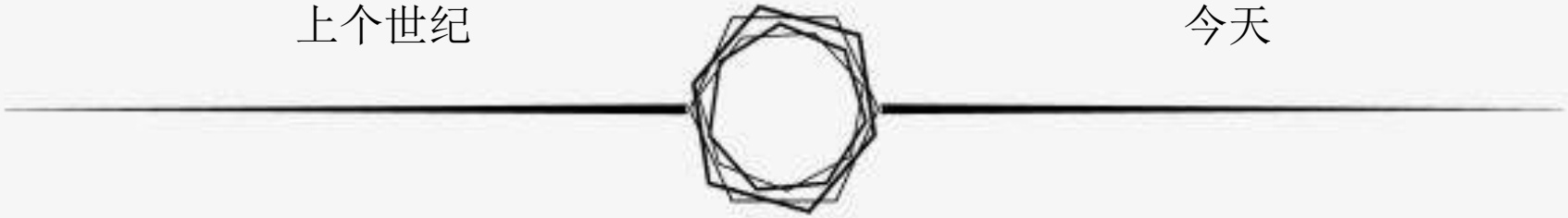
- 优点
  - 构件具有良好的隐蔽性和高内聚、低耦合的特点
  - 允许设计者将整个系统的输入/输出行为看成是多个过滤器的行为的简单合成
  - 支持软件重用：只要提供适合在两个过滤器之间传送的数据，任何两个过滤器都可被连接起来
  - 方便对一些如吞吐量、死锁等进行属性分析
  - 系统维护和增强系统性能简单：新的过滤器可以添加到现有系统中来；旧的可以被改进的过滤器替换掉
  - 支持并行执行：每个过滤器是作为一个单独的任务完成，因此可与其它任务并行执行

管道-过滤器风格

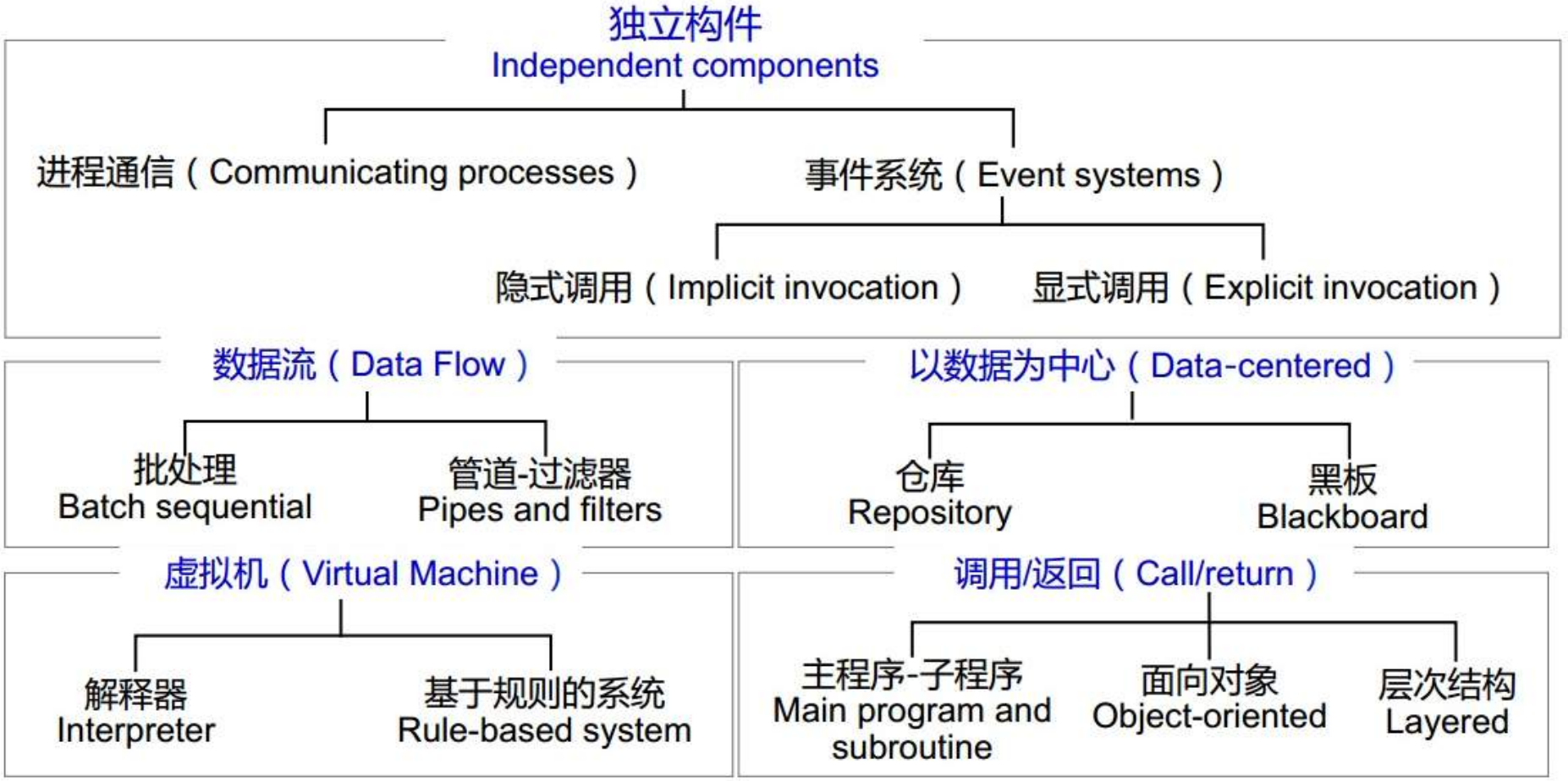
- 缺点
  - 通常导致进程成为批处理的结构
  - 不适合处理交互的应用
  - 在数据传输上没有通用的标准：每个过滤器都增加了解析和合成数据的工作，这样就导致了系统性能下降，并增加了编写过滤器的复杂性
  - 绝大部分处理时间消耗在格式转换上

软件体系结构风格

- 分割线



经典的体系结构风格


独立构件
Independent components
进程通信（Communicating processes）
事件系统（Event systems）
隐式调用（Implicit invocation）
显式调用（Explicit invocation）
数据流（Data Flow）
批处理
Batch sequential
管道-过滤器
Pipes and filters
以数据为中心（Data-centered）
仓库
Repository
黑板
Blackboard
虚拟机（Virtual Machine）
解释器
Interpreter
基于规则的系统
Rule-based system
调用/返回（Call/return）
主程序-子程序
Main program and subroutine
面向对象
Object-oriented
层次结构
Layered

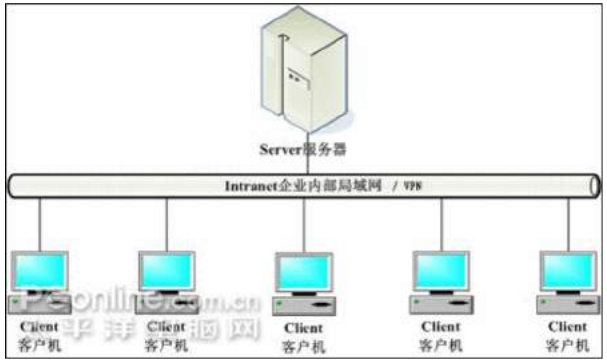
C/S和B/S风格

- 背景
  - 在集中式计算技术时代广泛使用的是大型机/小型机计算模型。它是通过一台物理上与宿主机相连接的非智能终端来实现宿主机上的应用程序
  - 20世纪80年代以后，集中式结构逐渐被以PC机为主的微机网络所取代。个人计算机和工作站的采用，改变了协作计算模型，导致了分散的个人计算模型的产生
  - C/S风格→ 三层C/S风格→ B/S风格

https://blog.csdn.net/linhaiyun_ytdx/article/details/72802522

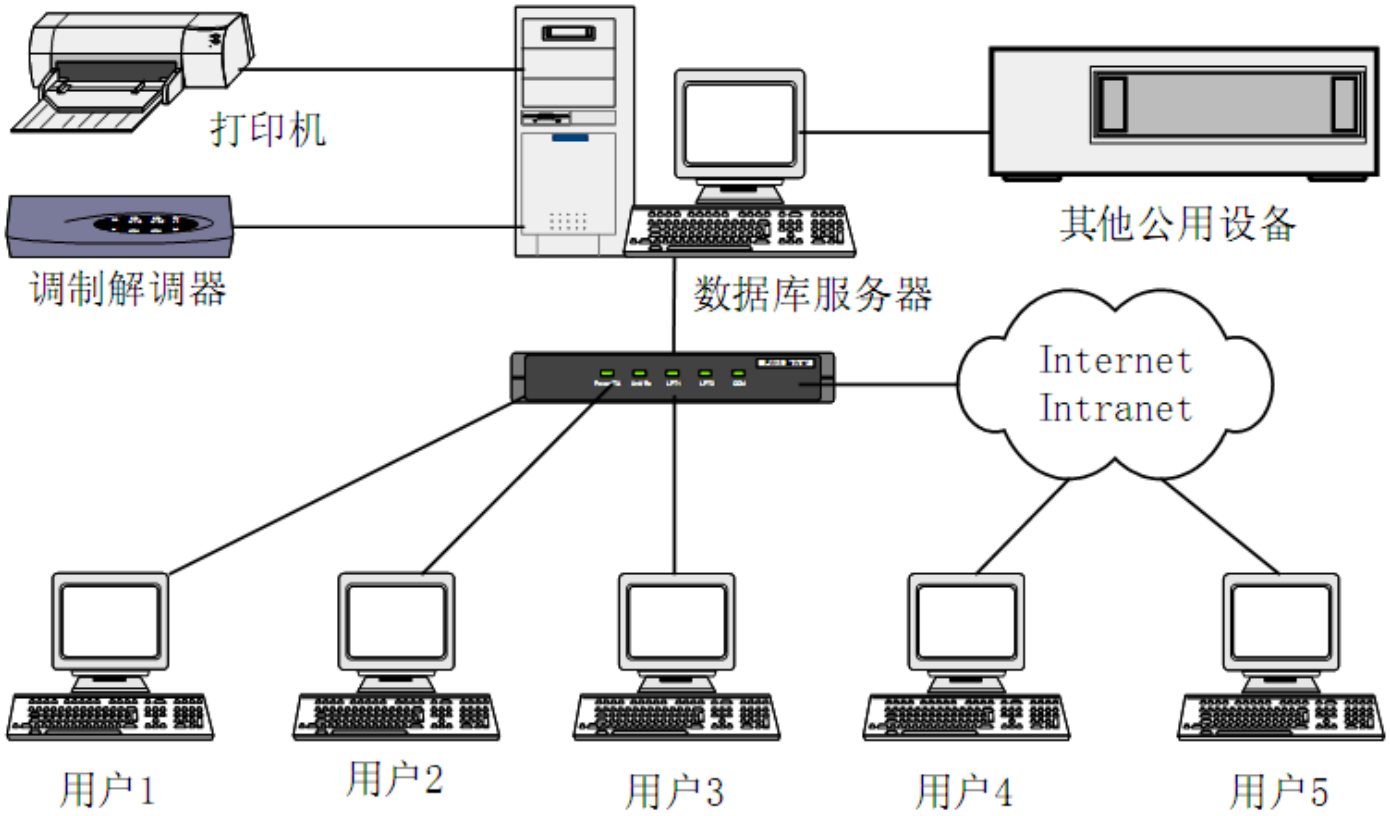
C/S风格

- 基本概念
  - C/S软件体系结构是基于资源不对等，且为实现共享而提出来的，是20世纪90年代成熟起来的技术，C/S体系结构定义了工作站如何与服务器相连，以实现数据和应用分布到多个处理机上
  - C/S体系结构有三个主要组成部分
    - 数据库服务器
    - 客户应用程序
    - 网络



C/S风格

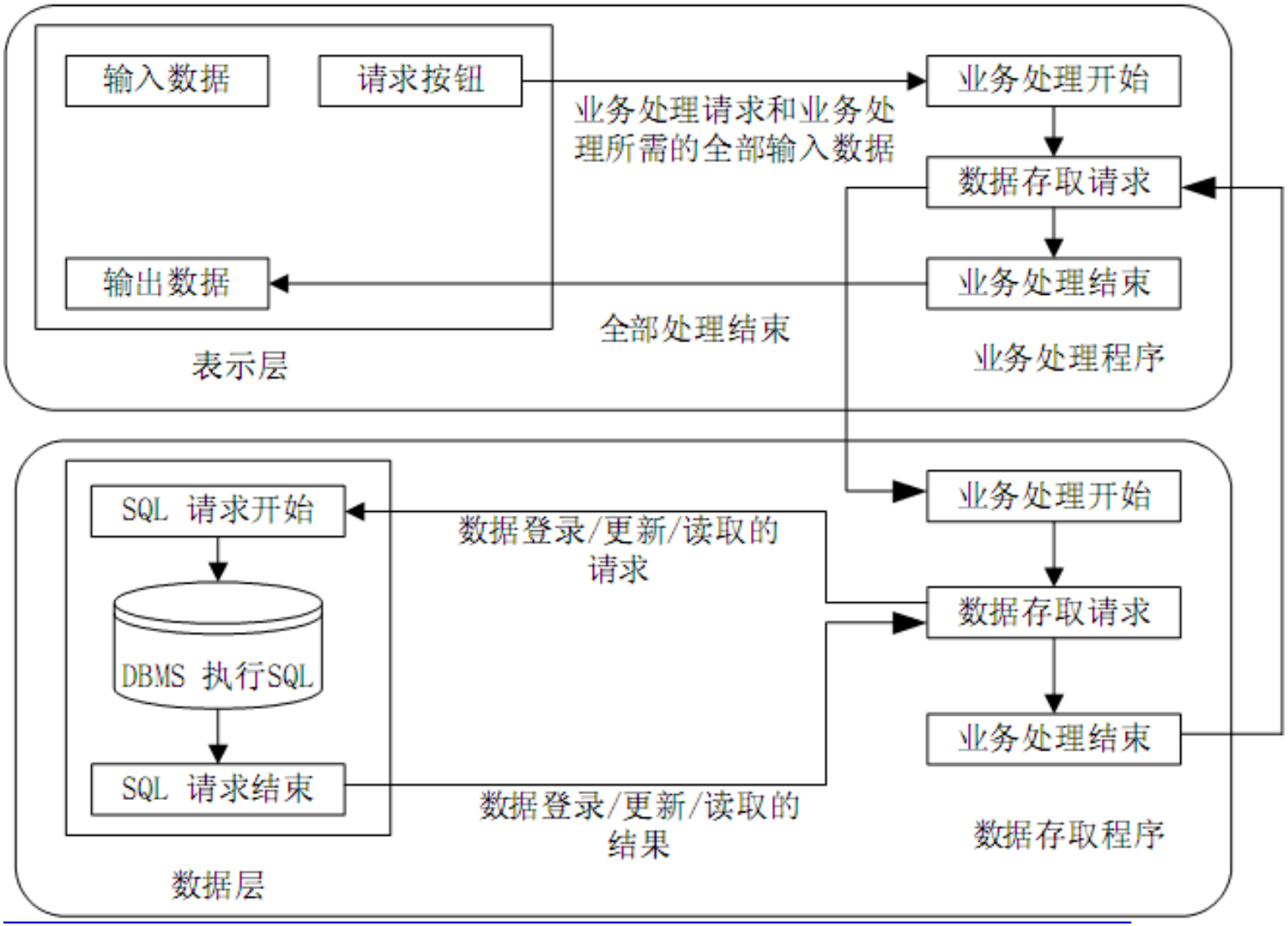
- 结构


打印机
调制解调器
数据库服务器
其他公用设备
Internet
Intranet
用户1
用户2
用户3
用户4
用户5

C/S风格

- 数据库服务器
  - 数据库安全性的要求
  - 数据库访问并发性的控制
  - 数据库前端的客户应用程序的全局数据完整性规则
  - 数据库的备份和恢复
- 客户应用程序
  - 提供用户与数据库交互的界面
  - 向数据库服务器提交用户请求并接收来自服务器的信息
  - 利用客户应用程序对客户端数据执行应用逻辑要求

C/S风格


输入数据
请求按钮
业务处理开始
业务处理请求和业务处理所需的全部输入数据
数据存取请求
输出数据
业务处理结束
全部处理结束
表示层
业务处理程序
SQL 请求开始
数据登录/更新/读取的请求
业务处理开始
数据存取请求
DBMS 执行SQL
业务处理结束
SQL 请求结束
数据登录/更新/读取的结果
数据层
数据存取程序

例子


小型超市零售系统
基础数据设置
业务信息管理
信息查询统计
采购商信息
商品信息
商品采购
商品销售
查询信息
统计信息
修改密码
重新登陆
退出系统


欢迎您使用图书管理系统
图书资源管理
借阅管理
读者管理
报表信息管理
权限管理
关于...
重新登陆
图书信息管理
图书采购管理
借书管理
还书管理
修改密码
退出系统
图书管理系统


MyQQ TCP方式
连接方式
TCP
UDP
服务选择
服务器
客户端
两 者
对方IP:
抓取屏幕
192 . 168 . 1 . 15
启 动
关 闭
你好HYB70-20121013E!
发送消息
传输文件
192.168.1.15->HYB70-20121013E于 20:30说道: 你好HYB70-
192.168.1.15<-HYB70-20121013E于 20:30说道: 你好HYB70-
停止操作
清 除
退出系统
番茄软件工作室 E-mail:ktdebug@163.com
PC6.com

C/S风格

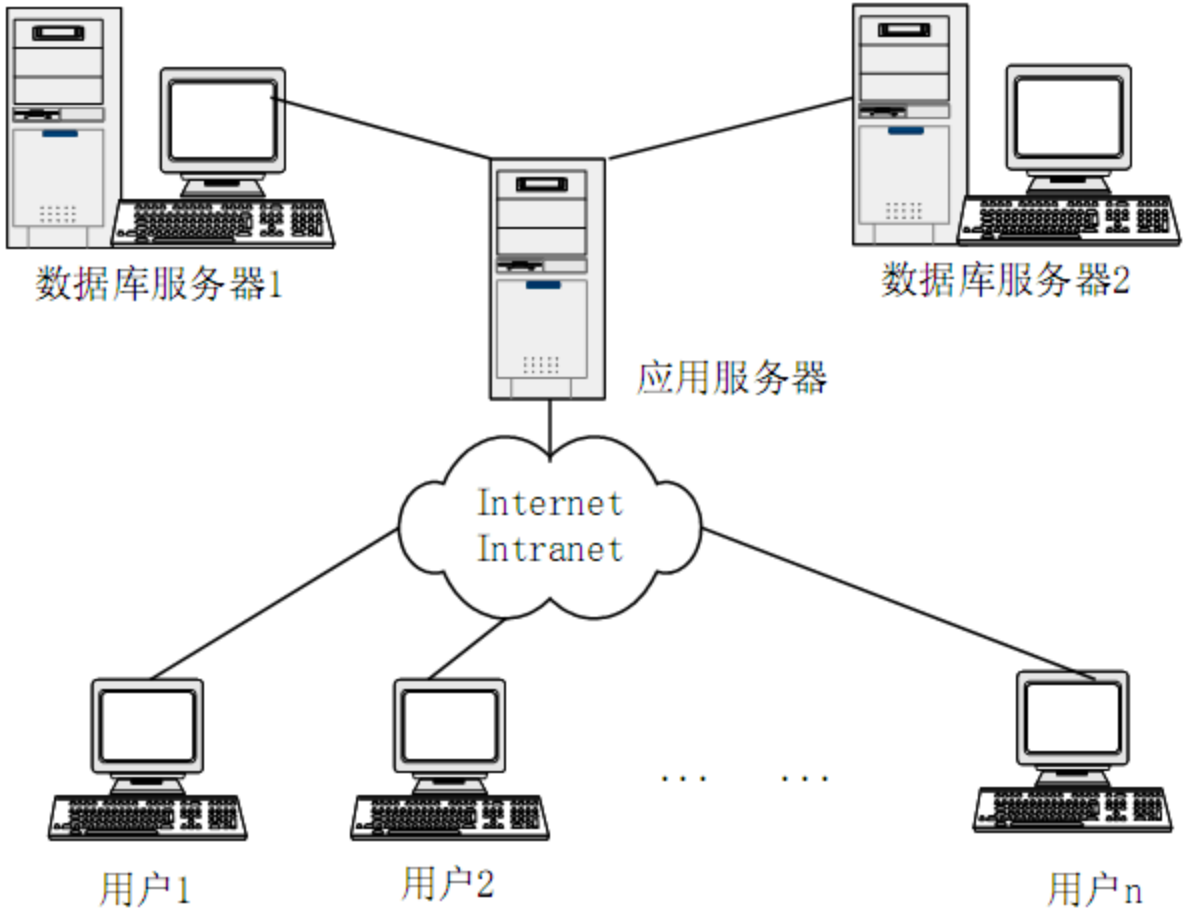
- 优点
  - 简单：具有强大的数据操作和事务处理能力， 模型思想简单，易于人们理解和接受。
  - 灵活：系统的客户应用程序和服务器构件分别运行在不同的计算机上，系统中每台服务器都可以适合各构件的要求，这对硬件和软件的变化显示出极大的适应性和灵活性，而且易于对系统进行扩充和缩小。
  - 低成本：系统中的功能构件充分隔离，客户应用程序的开发集中于数据的显示和分析，而数据库服务器的开发则集中于数据的管理。将大的应用处理任务分布到许多通过网络连接的低成本计算机上，以节约大量费用。

C/S风格

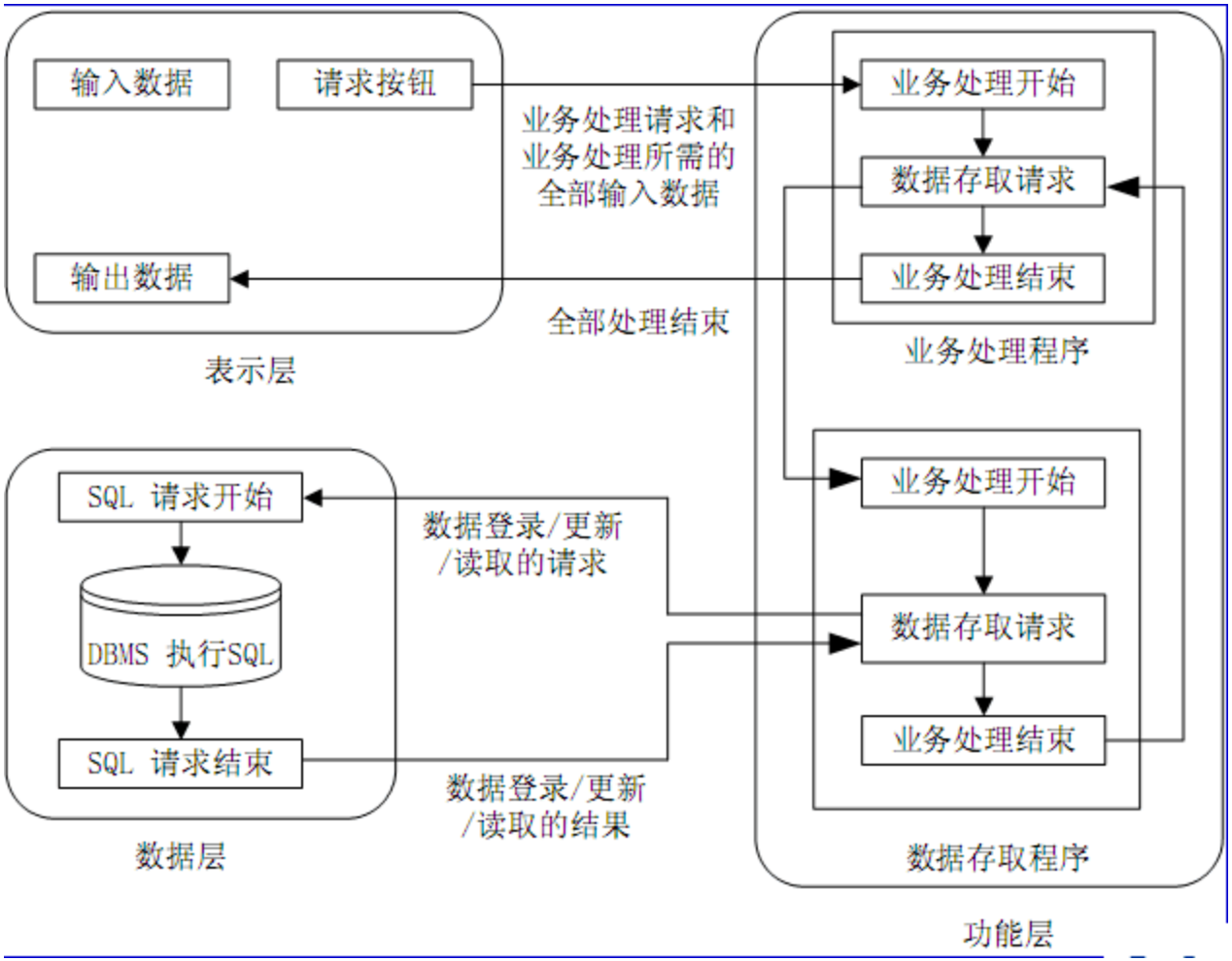
- 缺点
  - 开发成本较高
  - 客户端程序设计复杂
  - 信息内容和形式单一
  - 用户界面风格不一，使用繁杂，不利于推广使用
  - 软件移植困难
  - 软件维护和升级困难
  - 新技术不能轻易应用

三层C/S风格

- 结构



三层C/S风格


输入数据
请求按钮
输出数据
表示层
业务处理请求和
业务处理所需的
全部输入数据
全部处理结束
业务处理开始
数据存取请求
业务处理结束
业务处理程序
SQL 请求开始
DBMS 执行SQL
SQL 请求结束
数据层
数据登录/更新
/读取的请求
数据登录/更新
/读取的结果
业务处理开始
数据存取请求
业务处理结束
数据存取程序
功能层

三层C/S风格

- 实现


服务器2
数据层
服务器1
数据层
功能层
功能层
数据层
客户机
表示层
表示层
功能层
表示层
（1）将数据层和功能层放在同一台服务器上
（2）将数据层和功能层放在不同的服务器上
（3）将功能层放在客户机上

三层C/S风格

- 优点
  - 更清晰：合理地划分三层结构，使之在逻辑上保持相对独立性，提高系统和软件的可维护性和可扩展性
  - 更灵活：有效地选用相应的平台和硬件系统，使各个组成部分可以具有良好的可升级性和开放性；应用各层可并行开发，可以选择各自最适合的开发语言
  - 更安全：利用功能层有效地隔离开表示层与数据层，未授权的用户难以绕过功能层而利用数据库工具或黑客手段去非法地访问数据层，为严格的安全管理奠定了坚实的基础

三层C/S风格

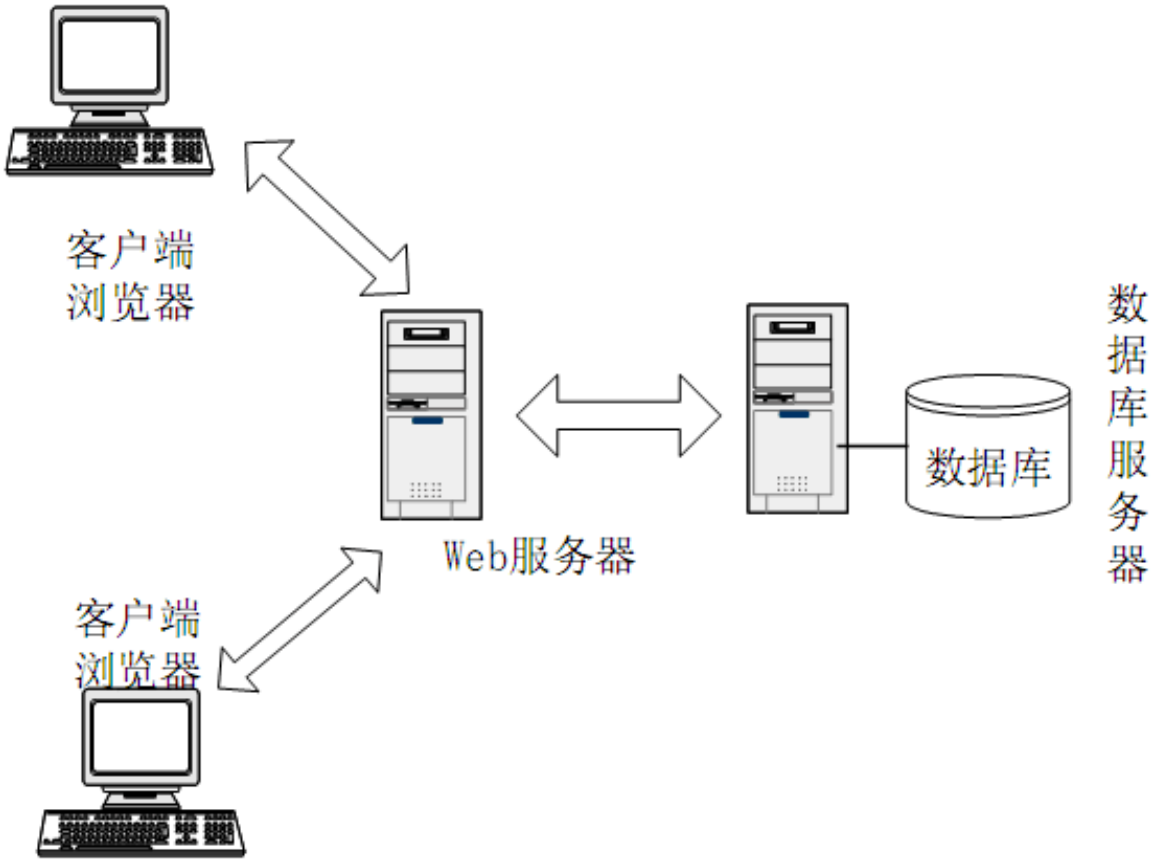
- 缺点
  - 通信效率低：即使分配给各层的硬件能力很强，其作为整体也可能达不到所要求的性能（如实时性）
  - 设计时必须慎重考虑三层间的通信方法、通信频率及数据量，这和提高各层的独立性一样是三层C/S结构的关键问题

B/S风格

- 定义：三层应用结构的一种实现方式，其具体结构为：浏览器 + Web服务器 + 数据库服务器
  - 利用不断成熟的浏览器技术，结合浏览器的多种脚本语言，用通用浏览器实现了原来需要复杂的专用软件才能实现的强大功能，并节约了开发成本。
  - 从某种程度上讲，B/S结构是一种全新的软件体系结构。

B/S风格

- 结构



B/S风格

- 优点
  - 易用性：基于B/S体系结构的软件，系统安装、修改和维护全在服务器端解决。用户在使用系统时，仅仅需要一个浏览器就可运行全部的模块，真正达到了“零客户端”的功能，很容易在运行时自动升级
  - 开放性：B/S体系结构还提供了异种机、异种网、异种应用服务器的联机、联网、统一服务的最现实的开放性基础

B/S风格

- 缺点
  - 安全性难以控制：风格攻击、脚本注入、代码泄漏...

C/S和B/S风格对比

角度	C/S	B/S
硬件环境	专用网络	广域网
安全要求	面向相对固定的用户群 信息安全的控制能力很强	面向是不可知的用户群 对安全的控制能力相对弱
程序架构	更加注重流程 系统运行速度可较少考虑	对安全以及访问速度要多重的考虑 B/S结构的程序架构是发展的趋势
软件重用	差	好
系统维护	升级难	开销小、方便升级
处理问题	集中	分散
用户接口	与操作系统关系密切	跨平台，与浏览器相关
信息流	交互性低	交互密集

C/S和B/S风格

- 案例分析

阅读以下关于 Web 应用开发的叙述，回答问题 1 至问题 3。

E-Mall 是一家电子商务公司，其主要业务是在线购物，包括书籍、服装、家电和日用品等。随着公司业务规模不断增大，公司决策层决定重新设计并实现其网上交易系统，公司负责系统开发的王工和李工分别给出了两种不同的设计方案，如下图 5-1 和图 5-2 所示。

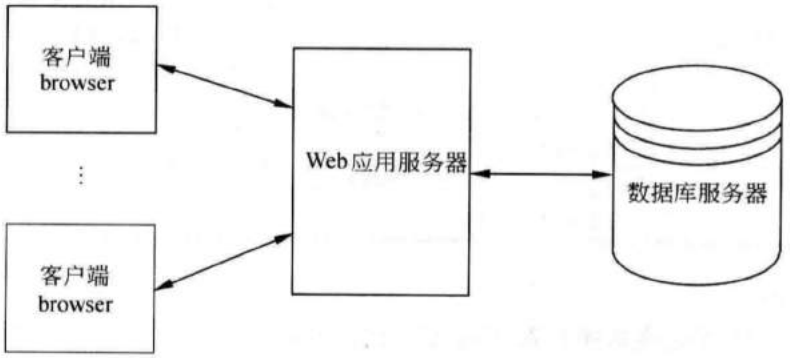


图 5-1　王工设计方案的体系结构设计示意图

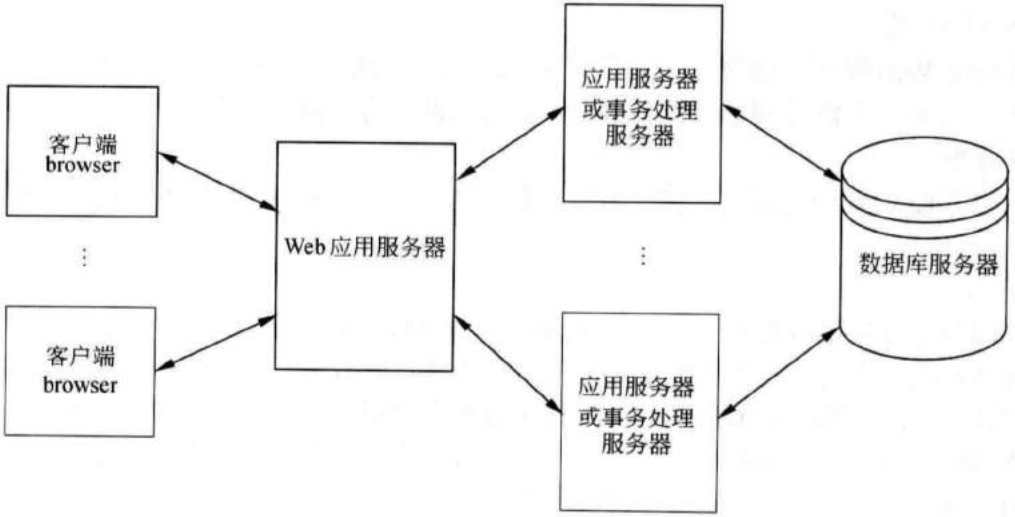


图 5-2　李工设计方案的体系结构设计示意图

公司的架构师和开发者针对这两种设计方案，从服务器负载情况、业务逻辑的分离性、系统可靠性、实现简单性等方面进行讨论与评估，综合考虑最终采用了李工给出的方案。

C/S和B/S风格

案例分析

【问题 1】（8 分）

请分析比较王工、李工两种方案的优点和不足，完成表 5-1 中的空白部分。

表 5-1

体系结构 / 方案 / 评价因素	王工建议的体系结构方案	李工建议的体系结构方案
服务器负载	Web 服务器需要同时处理业务逻辑与数据库访问，负担较重	（1）
业务逻辑的分离性	（2）	采用多个应用服务器专门进行业务逻辑处理，做到业务逻辑与其他代码分离
系统可靠性	采用单台 Web 服务器，整个系统的可靠性较差	（3）
实现简单性	主要采用 JSP、ASP 等脚本语言实现系统，比较简单	（4）

C/S和B/S风格

- 案例分析

参考答案

【问题 1】

体系结构方案 / 考虑因素	王工建议的体系结构方案	李工建议的体系结构方案
服务器负载	Web 服务器同时处理业务逻辑与数据库访问，负担较重	（1）Web 服务器处理用户请求，应用服务器处理业务逻辑与数据库访问，负载较为均衡
业务逻辑的分离性	（2）业务逻辑与数据库访问都位于 Web 服务器中。业务与逻辑没有分离	采用多个应用服务器专门进行业务逻辑处理，做到业务逻辑与其他代码分离
系统可靠性	采用单台 Web 服务器，整个系统的可靠性较差	（3）采用多台应用服务器，系统的可靠性较高
实现简单性	主要采用 JSP、ASP 等脚本语言实现系统，比较简单	（4）需要将脚本语言与面向对象编程语言相结合，相对复杂